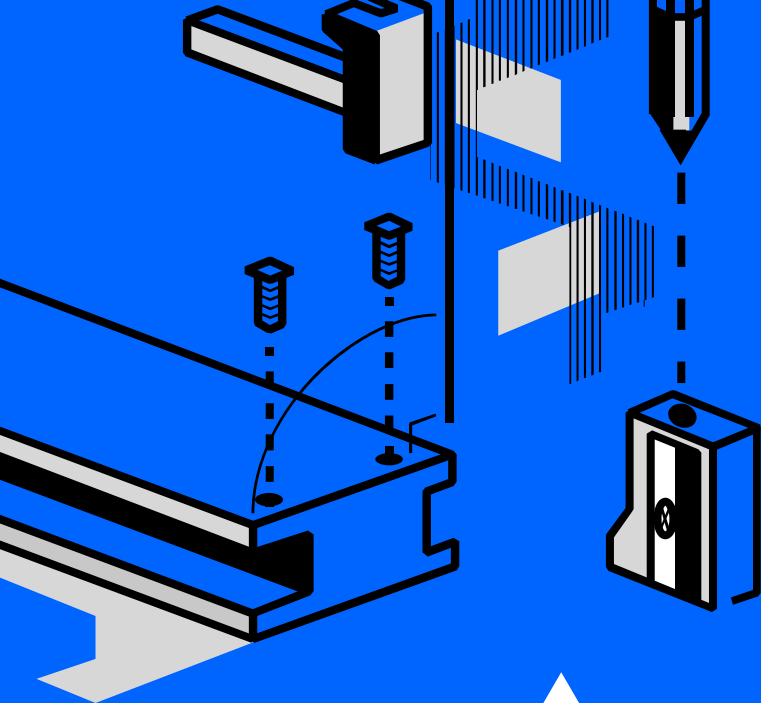


Full Stack- server

ראוטינג (Routing)

המכללה
הטכנולוגית
באר שבע

בשיתוף WORLD ORT
קדימה מדע



מה נלמד היום?

- מהו ראוטינג (ניתוב) ולמה הוא קריטי בשרת.
- מהו המבנה הבסיסי של הגדרת נתיב (Route) ב-Express.
- כיצד לטפל בכל ארבע פעולות ה - CRUD.

(GET, POST, PUT, DELETE)

- הבעיה שאנחנו יוצרים: מה קורה כשיש לנו יותר מדי נתיבים בקובץ אחד?

מהו ראוטינג? (הקונספט)



ראוטינג הוא ה"מרכזיה" או ה"מפה" של השרת.

זוהי הדרך שבה Express קובע כיצד להגיב לבקשת לקוח, בהתבסס על שני דברים:

1. **הנתיב (Path):** ה-URL שהלקוח ביקש (למשל: `/products/123/users`).
2. **המתודה (Method):** הפעולה שביקש (למשל: `GET`, `POST`).

אנלוגיה: אם השרת הוא בניין משרדים, הראוטינג הוא שילוט ההכוונה.

- בקשת `GET /users` אומרת: "קח אותי למשרד `users` (הנתיב) כדי לקרוא מידע (`GET`)".

המבנה הבסיסי ב-Express

הגדרת נתיב בסיסי ב-Express תמיד נראית כך:

```
app.METHOD(PATH, HANDLER)
```

- **app**: מופע האפליקציה של Express.
- **METHOD**: מתודת ה-HTTP באותיות קטנות (`get`, `post`, `put`, `delete`).
- **PATH**: הנתיב (URL) בשרת.
- **HANDLER**: הפונקציה שתרוץ כשהנתיב והמתודה תואמים. פונקציה זו תמיד מקבלת `(req, res)`.

דוגמה 1: ה-Route הכי פשוט

```
import express from 'express';  
const app = express();  
const port = 3000;  
  
// שלנו Route ה-  
app.get('/', (req, res) => {  
  // req - כל פרטי הבקשה (מהשיעור הקודם)  
  // res - האובייקט שמאפשר לנו לשלוח תשובה  
  res.send('Hello, World!');  
});  
  
app.listen(port, () => {  
  console.log(`Server listening on port ${port}`);  
});
```

מה קורה כאן? כשהדפדפן שולח בקשת GET לנתיב השורש (/), השרת מפעיל את הפונקציה ושולח בחזרה את הטקסט "Hello, World!".

דוגמה 2: טיפול ב-CRUD

בואו נדמיין שיש לנו מערך פשוט של משימות (tasks) ונבנה לו API שלם:

- GET /tasks - קבלת כל המשימות.
- POST /tasks - יצירת משימה חדשה.
- PUT /tasks/:id - עדכון משימה ספציפית.
- DELETE /tasks/:id - מחיקת משימה ספציפית.

דוגמת קוד: GET ו-POST

כדי לקרוא `req.body` אנחנו חייבים להשתמש ב-Middleware של `Express: app.use(express.json())`

```
// ... (app ו- app אחר הגדרת) ...  
let tasks = [  
  { id: 1, title: "Learn Routing", completed: true },  
  { id: 2, title: "Build an API", completed: false }  
];  
  
// GET - קבלת כל המשימות  
app.get('/tasks', (req, res) => {  
  res.json(tasks);  
});
```

```
// POST - יצירת משימה חדשה  
app.post('/tasks', (req, res) => {  
  // (body) מהשיעור הקודם: קוראים את הגוף  
  const newTask = {  
    id: tasks.length + 1,  
    title: req.body.title,  
    completed: false  
  };  
  tasks.push(newTask);  
  res.status(201).json(newTask);  
});
```

דוגמת קוד: DELETE ו-PUT

כאן נשתמש ב-`req.params` (שוב, מהשיעור הקודם) כדי לזהות איזו משימה לעדכן או למחוק.

```
// PUT - עדכון משימה לפי ID
app.put('/tasks/:id', (req, res) => {
  // מ-params מה ID-מהשיעור הקודם: קוראים את ה
  const taskId = parseInt(req.params.id);
  const task = tasks.find(t => t.id === taskId);

  if (task) {
    task.title = req.body.title || task.title;
    task.completed = req.body.completed !== undefined ?
req.body.completed : task.completed;
    res.json(task);
  } else {
    res.status(404).send('Task not found');
  }
});
```

```
// DELETE - מחיקת משימה לפי ID
app.delete('/tasks/:id', (req, res) => {
  const taskId = parseInt(req.params.id);
  tasks = tasks.filter(t => t.id !== taskId);
  res.status(204).send(); // 204 = No Content
});
```


הבעיה שיצרנו...

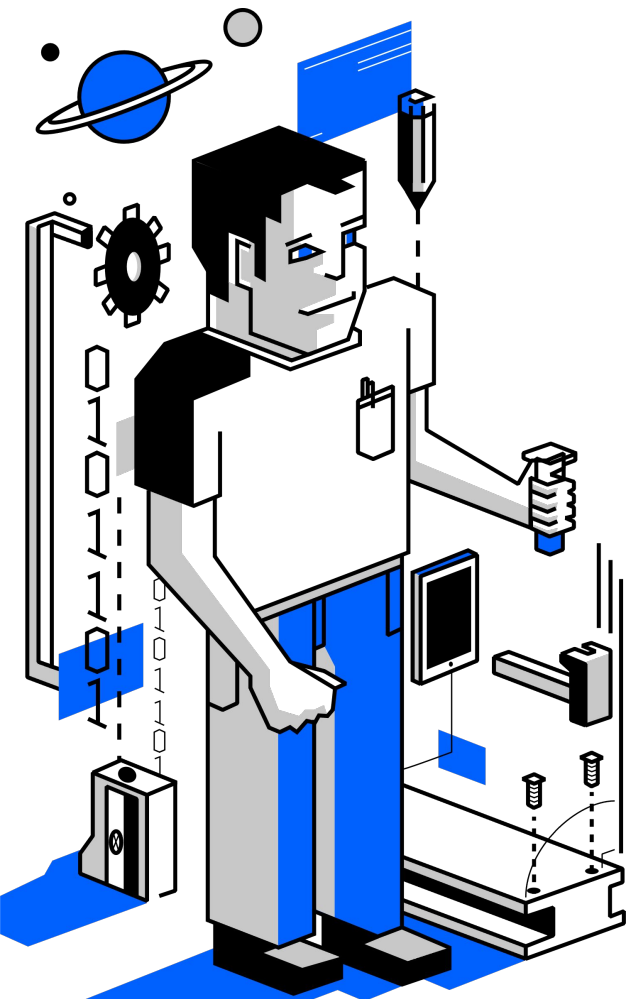
- הקוד שלנו עובד, אבל מה יקרה כשיש לנו 50 נתיבים?

```
// app.js - מתחיל להיות מבולגן...  
// --- User Routes ---  
app.get('/users', (req, res) => { /* ... */ });  
app.post('/users', (req, res) => { /* ... */ });  
app.put('/users/:id', (req, res) => { /* ... */ });  
  
// --- Product Routes ---  
app.get('/products', (req, res) => { /* ... */ });  
app.post('/products', (req, res) => { /* ... */ });  
app.put('/products/:id', (req, res) => { /* ... */ });  
  
// --- Order Routes ---  
app.get('/orders', (req, res) => { /* ... */ });  
app.post('/orders', (req, res) => { /* ... */ });  
// ... וכן הלאה ...
```

הקובץ `app.js` שלנו הופך ענקי, בלתי קריא ובלתי ניתן לתחזוקה.

סיכום והצעד הבא

- **סיכום:** ראוטינג הוא המנגנון של Express שמחבר בקשת לקוח (Method + Path) לפונקציה ספציפית (Handler).
 - **מה עשינו:** השתמשנו ב-`app.get`, `app.post`, `app.put`, `app.delete` כדי להגדיר נתיבים.
 - **הבעיה:** כל הלוגיקה שלנו נמצאת בקובץ אחד.
- בשיעור הבא:** נלמד איך לנקות את הבלגן הזה בעזרת מבנה פרויקט מודולרי ו-`express.Router`.



שאלות